```
      if (oldValue < value) {
        for (byte i=oldValue;  i<value; i++) {
          analogWrite(LED, i);
          delay(interval);
        }
      } else {
        for (byte i=oldValue;  i>value; i--) {
          analogWrite(LED, i);
          delay(interval);
        }
      }
      oldValue = value;
    }
    file.close();
    Serial.println("File closed.");
  } else {
    Serial.println("Error opening data.txt.");
  }
}
```

This example is a more complex version of the Flicker sketch presented in Chapter 8. Instead of using an array, we are reading sensor values from our memory card. This has the advantage that we do not need to worry so much about available memory space. Let's see how we read these values with the functions in the SD library.

available()

Because we may not know how many bytes are available in a particular file, we can use the `available()` function with the following syntax to find out:

```
name.available()
```

Okay, there's not much to it beyond giving it the name for our `File` instance, but it does return the amount of bytes that remain available in the file. When we open a file to read from it, we start at the beginning of the file, so we can use the `available()` function to determine when we have reached the end of the file. We used this function in the following manner:

```
while (file.available()) {
```

By placing the function in a `while` loop, we will continuously execute the following code so long as data is still available in the file that we are reading from.

read()

Now to get the information out of our file, we need to use the `read()` function. Like many of the other functions in this library, the syntax is minimal, as follows:

```
name.read()
```